Concept Graph Attention: Sequence Modeling over Fixed Concept Memory

Anonymous Author(s)
Affiliation
Address
email

Abstract

Self-attention stores one key-value entry per previous token and retrieves from that token table. This gives expressive content-addressed recall, but it ties the memory basis to token events and makes the read state grow with context length. We propose *Concept Graph Attention* (CGA), a causal sequence layer that keeps the continuous readout of attention but replaces token memory with a fixed bank of concept identities. Each concept has a fixed address and a runtime value memory. Tokens read from the concept bank with attention-style softmax weights and write back to the same bank with gated continuous updates and trainable retention.

CGA therefore changes where attention is computed: from token positions to a bounded concept memory whose contents evolve over the prefix. In the current formulation, the online state is only the concept value table, so the state size is CHd_v^c for C concepts, H heads, and per-head concept value width d_v^c. During training, gradients flow through the concept read, the continuous write, the write gate, and the retention parameters. During inference, model weights remain fixed, but the concept memory is updated by the same forward rule from the observed prefix. This gives CGA test-time learning of memory contents without an inference-time backward pass.

Empirically, we organize the evaluation around two ML-community baselines: state-equalized multi-query associative recall in the PoST/Zoology framework, and matched language-model pretraining on FineWeb-Edu at 180M–440M scale with standard zero-shot LM Harness evaluation. The intended comparison class is modern efficient attention alternatives such as Mamba, RWKV, Gated DeltaNet, and TTT-style stateful layers. CGA keeps attention-like retrieval, but moves the persistent state from token memory or learned test-time weights to a bounded, updateable concept memory.

1 Introduction

Attention can be read as a memory operation. Given a prefix, a Transformer stores one key-value entry per token and later queries retrieve a weighted combination of those token values. This token table is powerful because the read is continuous and differentiable, but it fixes the unit of memory: the model attends to token events. Long-context architectures have explored recurrent states, state-space models, linear memories, sparse attention, and inference-time optimizers. These approaches change the systems tradeoff, but the central memory question remains: can the model attend over a smaller set of learned runtime concepts instead of attending over every previous token?

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

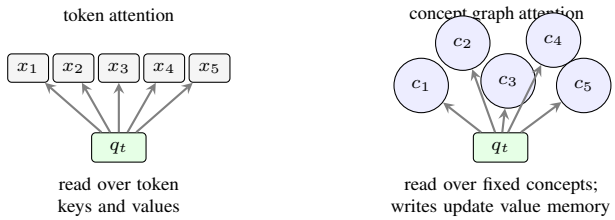

Figure 1: CGA changes the memory object of attention. Standard attention reads from token memory. CGA performs a continuous attention read over a fixed bank of concept identities whose value memories are updated causally.

We propose *Concept Graph Attention* (CGA). CGA keeps the familiar attention read,

$$\mathrm{read}(q_t) = \sum_{j=1}^{C} \mathrm{softmax}_j(s(q_t, c_{tj}))\, m_{tj},$$

but changes the objects being read. Instead of a token-indexed key-value table, each layer maintains a fixed bank of C concept slots. Each slot has a fixed concept identity and a value memory m_{tj} that evolves causally over the prefix. The read is attention over concepts; the write is a gated continuous update into the concept bank. Thus the layer is attention-like in its readout but recurrent-memory-like in its state:

$$\text{Transformer: tokens} \to \text{attention over token memory},$$
$$\text{CGA: tokens} \to \text{fixed concept memory} \to \text{attention over concepts}.$$

The fixed-bank design is deliberate. In language modeling and long synthetic recall, the data stream is effectively unbounded; a useful memory layer should eventually use its available capacity. CGA makes the capacity explicit from the first step and focuses the model on the harder problem: which concepts should be read and how should each token update them? The number of memory slots is a concept budget C rather than the context length T, giving a dense read cost of $O(TC)$ when $C \ll T$. At a fixed runtime-state budget, CGA can trade the number of concept slots against the per-slot value width.

This gives a different answer to the same question that motivates test-time-training sequence layers [Sun et al., 2025]. TTT layers make the hidden state a small model and update that model by a self-supervised learning step on the test sequence. CGA instead makes the hidden state a concept-indexed memory and updates that memory by a forward-only continuous write rule. Both mechanisms let a model adapt to information observed at inference time. The difference is the object being learned:

TTT: test-time learning of hidden model weights,
CGA: test-time learning of concept-memory contents.

We use "learning" here in the operational stateful sense: future predictions condition on a state estimated from the observed prefix. CGA does not update the global model parameters during inference and does not run an inference-time backward pass.

This paper makes three contributions.

- **Architecture.** We introduce fixed-bank Concept Graph Attention, a causal mixer with test-time concept learning: it reads continuously from concept slots and updates them with gated writes and trainable retention decay while model weights stay fixed at inference time.
- **Theory.** We formalize CGA as attention over an online concept value memory, specifying the state space, causal read/write equations, state-size scaling, block-scan update structure, and training boundary needed to compare it to recurrent, attention-like, and TTT-style sequence mixers.
- **Evaluation.** We implement CGA in the PoST/Zoology framework and evaluate it as an attention-like sequence mixer on two standard tracks: state-equalized multi-query associative recall and matched language-model pretraining with zero-shot downstream evaluation at 180M–440M scale.

The claim boundary is important. CGA is not presented as a new tokenizer, loss, or inference-time optimizer. It is a method proposal for changing the memory basis of attention from tokens to fixed runtime concepts. The current implementation trains slow projections by backpropagation, while the concept-memory update itself is a forward-time operation. This places CGA between attention, external memory, and efficient recurrent mixers: it uses differentiable content-addressed reads like attention, but the state being read is a compact, updateable concept memory rather than the entire prefix token table or a set of test-time-trained hidden weights.

2 Concept Graph Attention

CGA is an attention layer whose memory is a fixed bank of concept identities rather than a table of token keys and values. The "graph" is the runtime bipartite interaction between tokens and concept nodes, with soft read and write weights computed by attention. Each layer has C concept slots and H heads. For head h, slot j has a fixed unit address

$$a_{hj} \in \mathbb{R}^{d_a}, \qquad \|a_{hj}\|_2 = 1.$$

The addresses are initialized once and remain fixed; they are concept identities, not online sequence state. The online state for a sequence is the value memory

$$M_b \in \mathbb{R}^{H \times C \times d_v^c},$$

where b indexes a causal processing block and d_v^c is the per-head concept value width. Thus the persistent runtime state per layer is

$$S_{\mathrm{CGA}} = CHd_v^c.$$

This is the main state-size difference from token attention: attention stores a new key-value entry for every token, while CGA stores a bounded value table over fixed concept nodes.

Projection and normalization. Given hidden states $x_t \in \mathbb{R}^{d_{\mathrm{model}}}$, CGA forms normalized read queries and write keys, concept values, write gates, and retention rates:

$$q_{th} = \mathrm{normalize}(Q_h x_t), \qquad k_{th} = \mathrm{normalize}(K_h x_t),$$
$$v_{th} = V_h x_t, \qquad g_t = \sigma(w_g^\top x_t + b_g).$$

The query and key maps learn how tokens address the fixed concept identities. The value map learns what payload should be stored in concept memory, and the write gate learns how much a token should modify memory.

Dense concept read. CGA reads by applying the standard attention softmax over concept slots rather than over prefix tokens. Let $\gamma > 0$ be a learned read scale. For token t in block b,

$$\alpha_{thj} = \frac{\exp(\gamma q_{th}^\top a_{hj})}{\sum_{r=1}^{C} \exp(\gamma q_{th}^\top a_{hr})}, \qquad z_{th} = \sum_{j=1}^{C} \alpha_{thj} M_{b,hj}.$$

The multi-head output is

$$y_t = O[z_{t1}; \ldots; z_{tH}].$$

This is ordinary differentiable attention except that the keys are fixed concept identities and the values are runtime concept memories.

Continuous concept write. CGA uses a read-before-write protocol. The tokens in a block first read the memory state M_b; after those outputs are emitted, the block writes its values into memory to produce M_{b+1}. For a write delay $\delta \geq 0$, define

$$\bar{k}_{th} = k_{t-\delta,h},$$

with invalid delayed positions masked out. MQAR uses $\delta = 1$ so that a key token can bind to the following value token; general language modeling can use $\delta = 0$. The write routing distribution is another concept softmax:

$$\beta_{thj} = \frac{\exp(\gamma \bar{k}_{th}^\top a_{hj})}{\sum_{r=1}^{C} \exp(\gamma \bar{k}_{th}^\top a_{hr})}.$$

For block $b = [s, e)$, CGA updates each concept value by

$$M_{b+1,hj} = \rho_{bh} M_{b,hj} + \sum_{t=s}^{e-1} \mathbf{1}_{t \geq \delta}\, g_t\, \beta_{thj}\, v_{th}.$$

The write is continuous: every valid token can write to every concept with a different weight, and the soft weights make the write path differentiable during training.

Trainable retention. The factor $\rho_{bh} \in (0, 1]$ controls how much old concept memory survives when a new block is written. CGA computes tokenwise positive decay rates λ_{th} and accumulates them over the block:

$$\rho_{bh} = \exp\left(-\sum_{t=s}^{e-1} \lambda_{th}\right).$$

Our default parameterization uses a PoST-style ordered spectrum for the base decay rates:

$$A_h = A_0 + \sum_{i<h} \mathrm{softplus}(\Delta_i),$$

combined with a learned input-dependent step size and optional position-adaptive scaling. This gives CGA the same kind of trainable forgetting mechanism used by modern recurrent mixers, but the recurrence acts on concept value memory rather than on token-indexed state.

Training and inference. During training, CGA is optimized end-to-end with the task loss. Gradients flow through the read weights α, the write weights β, the written values v_t, the write gate g_t, and the retention factors ρ_{bh}. The concept addresses a_{hj} are fixed identities, so the model learns to use them by changing the query, key, value, gate, retention, and output projections. During inference, the learned parameters are fixed. The concept memory M_b is still updated by the same forward rule, so future tokens condition on information written from the observed prefix. This is the sense in which CGA performs test-time learning: it learns the contents of a bounded memory during the forward pass, without running backpropagation at inference time.

Relation to ordinary attention. Standard causal attention computes

$$\mathrm{Attn}(X)_t = \sum_{s<t} \mathrm{softmax}_s(q_t^\top K_s) V_s,$$

where the memory basis is the set of prefix token positions. CGA computes

$$\mathrm{CGA}(X)_t = \sum_{j=1}^{C} \mathrm{softmax}_j(q_t^\top a_j) M_{tj},$$

where the memory basis is a fixed set of concept identities and only the value contents M_{tj} evolve over the prefix. Thus CGA is not a different loss or optimizer around attention. It is an attention replacement that moves the read/write memory from tokens to concepts.

3 Experiments

We evaluate CGA as a causal sequence mixer: a layer with the external contract of attention, but whose memory is a bounded concept bank rather than all previous tokens. The empirical program has two tracks. First, multi-query associative recall (MQAR) isolates the key mechanism: writing key-value information into runtime memory and retrieving it later under matched state budgets. Second, matched language-model pretraining tests whether the same concept read/write layer remains useful in ordinary next-token prediction. The LM protocol follows the PoST paper's evaluation style so that CGA can be compared directly against Mamba-2, RWKV-7, and Gated DeltaNet runs trained on the same data and evaluated with the same harness.

Algorithm 1 CONCEPTGRAPHATTENTIONFORWARD

Input: hidden states $x_{1:T}$, fixed concept addresses $a_{1:C}$, block size B.
Initialize value memory $M_0 \leftarrow 0$.
for causal blocks $b = [s, e)$ **do**
  Project block queries q, write keys k, values v, gates g, and retention rates λ.
  Read M_b with dense concept attention over fixed addresses.
  Emit block outputs through the output projection.
  Form delayed write keys $\tilde{k}$ and write weights β.
  Compute block retention $\rho_b = \exp(-\sum_{t=s}^{e-1} \lambda_t)$.
  Update $M_{b+1} \leftarrow \rho_b M_b + \sum_{t=s}^{e-1} g_t \, \beta_t \, v_t$.
end for
Return projected concept readouts.

3.1 Experiment 1: Multi-Query Associative Recall

MQAR inputs contain key-value pairs followed by queries; the model must output the corresponding value [Arora et al., 2024]. This is the cleanest test for CGA because the task separates two questions: whether the model can store associations in a runtime memory, and whether later queries can retrieve them. For token attention, the memory is the raw prefix. For CGA, the memory is a fixed concept bank whose value contents are written from the prefix.

We run CGA inside the existing PoST/Zoology MQAR framework. The comparison is state-equalized against existing efficient sequence mixers. We use the same grid as the PoST/Mamba benchmark runs:

$$(d_{\text{model}}, H) \in \{(512, 4), (512, 8), (256, 4)\}, \qquad d_{\text{state}} = d_{\text{model}}/(2H).$$

These correspond to target runtime-state budgets $d_{\text{model}}^2/H \in \{64\text{K}, 32\text{K}, 16\text{K}\}$. For MQAR, a concept slot is intended to hold an association, so the target concept capacity is tied to the largest training key-value count rather than to the sequence length. The curriculum reaches 128 key-value pairs, so the main grid sets $C = 128$ for every CGA configuration.

For each grid point we choose a per-head concept value bottleneck d_v^c so that the persistent CGA state

$$S_{\text{CGA}} = CHd_v^c$$

does not exceed d_{model}^2/H. This is the runtime value state stored by fixed-bank CGA; concept identities are fixed layer parameters and are not counted as online sequence state. The value bottleneck is rounded down to an eight-dimensional alignment. This gives $(C, d_v^c) = (128, 128)$ for $(512, 4)$, $(128, 32)$ for $(512, 8)$, and $(128, 32)$ for $(256, 4)$, matching the 64K, 32K, and 16K state budgets. Each grid point is run with the same three-point learning-rate sweep

$$\{10^{-2.5}, 10^{-2}, 10^{-1.5}\}.$$

The implementation uses dense concept reads, untied read/write address projections, a learned read logit scale initialized at the standard dot-product attention scale, a learned write gate with sparsity coefficient 10^{-4}, fixed concept identities, and continuous value writes. The main run uses the PoST parameterization for the retention spectrum, with $\mathrm{dt} \in [10^{-6}, 3 \cdot 10^{-5}]$, train length 512, and position-adaptive scaling enabled. Training uses sequence length 512 with the same curriculum over key-value count as the PoST baselines. Evaluation reports the existing persist and capacity slices at lengths $512, 1024, 2048, 4096$.

Baselines. The comparison class is the standard MQAR suite already present in the lab framework:

- **Transformer**: token-level causal self-attention.
- **Mamba/RWKV/GDN/GLA/RetNet**: recurrent or linear-state efficient mixers.
- **TTT-style layers**: expressive state updated by test-time learning style mechanisms.
- **CGA**: attention over a fixed concept memory.

What the experiment tests. The fair-state run asks whether replacing token memory by concept memory is competitive at the same state budget. The important diagnostic slices are:

$$(\text{test length},\ \text{key-value count},\ \text{persist/capacity}).$$

A positive result is not merely low training loss. CGA should maintain recall when the query distance grows, expose a compact runtime state, and report latency and concept count so that improvements are not hidden inside unbounded memory. We additionally log the mean write gate, valid-token write gate, write mass per token, read scale, mean retention, and mean decay rate. These diagnostics are part of the main run because the central failure mode for CGA on MQAR is not only optimization loss, but whether useful key-value writes are routed into the fixed concept bank and whether the retention time constants are compatible with the sequence length.

Table 1: **MQAR capacity accuracy (%)** at each test length T with $K{=}T/4$ key-value pairs. All models are trained at $T{=}512$; longer lengths are out-of-distribution. Baseline and PoST rows reproduce the state-equalized PoST/Zoology MQAR table. The CGA row is reserved for the current fixed-bank CGA run.

	state= $64K$					state= $32K$					state= $16K$				
Model	512	1K	2K	4K	**Avg**	512	1K	2K	4K	**Avg**	512	1K	2K	4K	**Avg**
Mamba-2	100.0	96.8	62.2	18.9	69.5	99.2	85.2	41.3	11.6	59.4	99.3	80.6	31.2	5.7	54.2
+PoST	100.0	97.4	68.3	25.1	72.7	99.8	92.1	51.6	13.2	64.2	99.6	87.8	44.1	12.7	61.0
RWKV-7	100.0	100.0	96.1	39.2	83.8	100.0	100.0	80.1	9.5	72.4	100.0	95.2	46.0	10.8	63.0
+PoST	100.0	100.0	98.5	52.9	87.8	100.0	100.0	98.0	28.5	81.6	100.0	99.3	70.9	18.8	72.2
Gated DeltaNet	100.0	100.0	92.0	42.4	83.6	100.0	96.4	56.7	15.9	67.2	99.8	82.7	31.7	7.4	55.4
+PoST	100.0	100.0	95.3	48.4	85.9	100.0	99.9	88.9	39.6	82.1	99.9	86.5	35.7	8.7	57.7
GLA	100.0	97.8	67.2	20.8	71.5	100.0	97.7	50.3	7.6	63.9	99.8	88.5	38.7	7.8	58.7
+PoST	100.0	96.0	62.1	20.7	69.7	99.9	93.9	54.8	16.9	66.4	99.9	93.1	50.7	12.2	64.0
RetNet	99.9	47.1	2.3	0.0	37.3	99.9	63.2	6.0	0.3	42.3	96.8	16.8	0.7	0.0	28.6
+PoST	100.0	96.0	62.1	20.7	69.7	99.9	93.9	54.8	16.9	66.4	99.9	93.1	50.7	12.2	64.0
CGA (ours)	**100.0**	**100.0**	**100.0**	**90.0**	–	–	–	–	–	–	–	–	–	–	–

The implementation artifact is `../PoST_dev/zoology/zoology/experiments/cga_mqar.py`. The registered mixer is `zoology.mixers.cga.ConceptGraphAttention`.

3.2 Experiment 2: Language-Model Pretraining and Evaluation

The language-model experiment is the scaling test. It asks whether fixed concept memory can replace token attention inside a standard decoder and remain competitive with modern efficient recurrent mixers. We follow the PoST language-model protocol: pretrain base models on FineWeb-Edu [Lozhkov et al., 2024] at context length $T_{\text{train}} = 2048$, then evaluate zero-shot with the Language Model Evaluation Harness [Gao et al., 2024]. The primary scale is 180M parameters; a 440M run is the direct scaling check on $8\times$H200 hardware.

CGA-LM model. CGA-LM uses the same decoder scaffold as the baseline language models: token embeddings, residual blocks, normalization, MLPs, and tied LM head are standard. Only the causal mixer is changed: the attention or recurrent mixing sublayer is replaced by CGA. For a state-matched comparison to the PoST Mamba-2 configurations, we set

$$C = 128, \qquad d_v^c = 64, \qquad S_{\text{CGA}} = CHd_v^c.$$

At 180M, using $H = 24$ gives $S_{\text{CGA}} = 196{,}608$ values per layer, matching the Mamba-2 state budget 1536×128. At 440M, using $H = 32$ gives $S_{\text{CGA}} = 262{,}144$, matching 2048×128. Concept identities are fixed layer parameters and are not counted as online runtime state. The main CGA-LM run uses fixed concept identities, dense concept reads, continuous value writes, a learned write gate, and PoST-style retention parameterization.

Metrics. We report validation loss and perplexity during pretraining, then downstream zero-shot results on the same task set as PoST: LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, WinoGrande, and OpenBookQA. For systems comparison, each run should also report training tokens per second, prefill latency, decode latency, parameter count, online state size, concept count, mean write gate, read scale, and retention statistics. The speed numbers are not secondary: CGA's claim is

Table 2: **Downstream zero-shot language-model evaluations.** Baseline and PoST rows reproduce the PoST LM evaluation table. CGA-LM rows are reserved for fixed-bank CGA runs trained with the matched FineWeb-Edu protocol and evaluated with the same LM Harness tasks.

Model	LAMBADA		HellaSwag	PIQA	ARC-Easy	ARC-Challenge	WinoGrande	OpenBookQA	Avg
	acc↑	ppl↓	acc_n ↑	acc↑	acc↑	acc_n ↑	acc↑	acc_n ↑	
Mamba-2 180M	**21.6**	**145.4**	31.1	62.9	**50.4**	24.7	49.6	**30.6**	38.7
+PoST	21.5	148.2	**31.3**	**63.2**	50.1	**24.9**	**50.6**	30.0	**38.8**
RWKV-7 180M	27.9	**69.6**	32.1	**63.1**	49.7	**25.7**	51.3	29.0	39.8
+PoST	**28.3**	71.9	32.1	62.9	**52.1**	25.3	**51.8**	**32.0**	**40.6**
Gated DeltaNet 180M	23.8	**94.5**	**31.9**	62.7	49.6	24.2	**51.1**	30.6	39.1
+PoST	**25.2**	95.6	31.5	**62.9**	**51.5**	**25.3**	50.7	**31.8**	**39.8**
CGA-LM 180M (ours)	–	–	–	–	–	–	–	–	–
Mamba-2 440M	24.1	77.3	**37.7**	65.3	**57.7**	**27.2**	50.4	**32.8**	42.2
+PoST	**28.0**	**62.6**	37.5	65.3	56.6	26.2	**51.4**	32.6	**42.5**
CGA-LM 440M (ours)	–	–	–	–	–	–	–	–	–

an attention-compatible memory replacement, so the correct comparison includes both quality and sequence-processing cost.

The main reported comparison is the best CGA-LM checkpoint at each scale against the best baseline row at the same scale. We do not need a broad ablation suite for the first large run. The decisive experiment is whether one clean CGA-LM configuration can match or exceed these baseline averages while using a bounded concept memory and competitive latency.

4 Related Work

Efficient sequence mixers and expressive state. Transformers use content-addressed attention over previous tokens [Vaswani et al., 2017]. Mamba, RWKV, GDN, GLA, RetNet, and related models replace attention with efficient recurrent or linear state updates [Gu and Dao, 2023, Peng et al., 2023, Yang et al., 2025, 2024, Sun et al., 2023]. TTT-style models instead make the hidden state expressive by running a learned update rule at test time [Sun et al., 2025]. Our experiments place CGA in this comparison class: CGA is an attention-like causal mixer whose persistent state is a bounded concept memory rather than a token table.

Test-time learning layers. TTT layers frame sequence modeling as test-time learning: the hidden state is a small model, and the state update is a self-supervised training step on the current sequence [Sun et al., 2020, 2025]. CGA adopts the same high-level goal, namely to let inference-time computation learn from the observed prefix, but changes the learning object. The hidden state in CGA is not a linear model or MLP whose weights are updated; it is a concept-indexed value memory. The update is therefore forward-only at inference: read concepts, decay old values, and add gated continuous writes. This makes CGA complementary to TTT. TTT explores expressive hidden states as trainable inner models; CGA explores expressive hidden states as writable concept memories with attention-style readout.

Attention variants. Sparse attention, sliding-window attention, grouped-query attention, and compressed-memory attention mainly alter which token keys and values are read or how the token table is stored. CGA is orthogonal: it changes the basis being attended to. The read still has the same softmax form, so optimized attention kernels and attention variants can in principle be adapted to concept slots instead of token positions.

Associative and external memory. Cache language models, kNN-LMs, retrieval-augmented systems, fast weights, online ridge or RLS memories, Memory Networks, Neural Turing Machines, and Differentiable Neural Computers all use forms of stateful retrieval or local memory update [Grave et al., 2017, Khandelwal et al., 2020, Lewis et al., 2020, Ba et al., 2016, Schlag et al., 2021, Crammer et al., 2006, Weston et al., 2015, Graves et al., 2014, 2016]. CGA shares the associative-memory goal but keeps the decoder-block interface of attention: each token reads from concept slots and writes back through a gated causal update.

Routing and capacity. Mixture-of-experts, Switch Transformers, and Product-Key Memory route tokens or queries to trained capacity [Shazeer et al., 2017, Fedus et al., 2022, Lample et al., 2019]. CGA also routes writes, but the routed capacity is runtime value state rather than a bank of trained expert parameters. The fixed concept count C makes this capacity explicit and state-equalizable against recurrent mixers.

5 Limitations and Conclusion

- CGA's dense read is stable at small concept budgets, but large concept banks will need fused kernels and eventually sparse concept retrieval.
- The current retention decay is trainable and PoST-parameterized by default, but its best time-scale allocation for large language modeling remains empirical.
- Fixed-bank CGA removes online slot-allocation complexity, but it makes the concept count C a real capacity hyperparameter.
- CGA's test-time learning is prefix-local in the current formulation: the concept value memory is reset per sequence unless a separate persistent-memory protocol is added.
- The 180M–440M language-model runs are the intended SOTA-readiness test, but they still require completed training and latency reporting before the paper can claim a large-scale win.

The empirical claim is deliberately scoped: CGA is evaluated as an attention-like causal sequence mixer on MQAR and matched language-model pretraining. MQAR should show whether the concept bank can write and retrieve key-value information under matched state budgets. CGA-LM should show whether the same mechanism can replace token attention in a trainable decoder at 180M–440M scale. If CGA loses to optimized recurrent mixers, that is a real systems or modeling gap rather than a wording problem. The method still needs stronger fused kernels, retention-time-scale studies, and distributed concept-bank implementations before it can be treated as a fully optimized large-model systems result.

The main point is the memory-basis replacement. Attention reads from token-indexed memory. CGA reads from fixed concept identities whose value memories are updated by a causal forward-time rule during inference. This gives a clean attention-compatible path for testing whether concept-level test-time learning can compete with token memory, recurrent state, and TTT-style hidden-model updates under matched state budgets.

References

Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*, 2024.

Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, et al. A framework for few-shot language model evaluation. https://github.com/EleutherAI/lm-evaluation-harness, 2024.

Edouard Grave, Moustapha M. Cisse, and Armand Joulin. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626): 471–476, 2016. doi: 10.1038/nature20101.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Herve Jegou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. FineWeb-Edu: The finest collection of educational content the web has to offer. *Hugging Face Blog*, 2024. https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, et al. RWKV: Reinventing RNNs for the transformer era. *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *International Conference on Machine Learning*, 2024.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

Appendix

A CGA-LM: Replacing Token Attention with Concept Graph Attention

For language modeling, the world state is a prefix and the consequence is the next token. CGA-LM replaces the causal attention sublayer inside a decoder block with Concept Graph Attention:

$$\mathrm{CGABlock}(X) = X + \mathrm{CGA}(\mathrm{Norm}(X)).$$

The block has the same external shape contract as attention, $[B, L, d_{\mathrm{model}}] \mapsto [B, L, d_{\mathrm{model}}]$. The surrounding MLP, residual stream, layer normalization, embeddings, and language-model head are otherwise standard decoder components.

A.1 Fixed concept memory

At layer ℓ, CGA-LM keeps a fixed bank of C concept identities $a_{\ell hj} \in \mathbb{R}^{d_a}$ for each head h and concept slot j. These identities are fixed addresses shared across sequences. The online state is the value memory

$$M_t^\ell \in \mathbb{R}^{H \times C \times d_v^c}.$$

Each token produces read queries, write keys, write values, write gates, and retention-decay projections:

$$q_t = Q_\ell x_t, \qquad k_t = K_\ell x_t, \qquad v_t = V_\ell x_t, \qquad g_t = \sigma(w_g^\top x_t + b_g).$$

The read is causal: token t reads only concept values produced by earlier writes. For concept slots $j = 1, \ldots, C$,

$$\alpha_{thj} = \mathrm{softmax}_j \left(\gamma_\ell q_{th}^\top a_{\ell hj}\right), \qquad z_{th} = \sum_{j=1}^{C} \alpha_{thj} M_{t,hj}^\ell, \qquad y_t = O_\ell[z_{t1}; \ldots; z_{tH}].$$

Multi-head CGA uses independent address and value subspaces per head and concatenates the head outputs before the output projection, exactly as multi-head attention does.

A.2 Read before write

The update order is the same during training and autoregressive inference:

$$\text{read from } M_t^\ell \to \text{emit } y_t \to \text{decay and write } (k_t, v_t, g_t) \to \text{update } M_{t+1}^\ell.$$

This prevents a token from reading its own write. The inference-time update is the test-time learning mechanism in CGA-LM: the model weights are fixed, but the concept value memory changes after the causal write and future tokens condition on that updated memory.

For a delayed write key $\bar{k}_t = k_{t-\delta}$, CGA computes write weights

$$\beta_{thj} = \mathrm{softmax}_j \left(\gamma_\ell \bar{k}_{th}^\top a_{\ell hj}\right).$$

With retention factor ρ_{th}, the token-level streaming update is

$$M_{t+1,hj}^\ell = \rho_{th} M_{t,hj}^\ell + g_t \beta_{thj} v_{th}.$$

For parallel training, the implementation groups tokens into causal blocks: all tokens in a block read the memory from before the block, then their writes are accumulated into one block update. Setting block size one recovers the exact token-streaming update.

Both MQAR and language-model variants use a learned write gate. The current experiments also use a trainable retention factor with a PoST-style ordered decay parameterization. This is a parameterization choice for the forgetting spectrum, not the definition of CGA itself; CGA can be run with a free retention parameterization for ablations.

A.3 Training boundary

Slow neural parameters are trained by ordinary next-token backpropagation through both the CGA read and the continuous write. Gradients flow through the read weights, write weights, value projection, write gate, retention factors, and output projection. At inference time there is no gradient step and no update to model weights. The only state that changes is the forward-updated concept value memory. Thus CGA-LM is not a backprop-free language model and not a test-time optimizer over hidden model weights. It is an attention replacement whose fast state is a bounded concept memory.

Algorithm 2 CGALAYERFORWARD

```
Input: hidden states x_{1:L}, fixed concept identities a_{1:C}, zero value memory M_0
for causal blocks do
    project block queries, write keys, values, gates, and retention rates
    read fixed concept memory with continuous attention weights
    emit block outputs through the output projection
    form delayed write keys and write weights
    decay old value memory and add gated continuous writes
end for
return contextualized hidden states
```

A.4 Scaling path

The reference CGA implementation uses dense concept reads. This is appropriate for state-equalized MQAR and for the first language-model runs with $C = 128$. The large-scale implementation target is a fused dense CGA kernel that combines concept-score computation, softmax, value read, retention, and gated write accumulation inside the mixer. For substantially larger concept banks, sparse concept retrieval and sharded concept banks become systems work. The paper therefore separates the architectural claim from the kernel claim: CGA changes the memory basis of attention from tokens to concepts; optimized implementations should then choose dense or sparse concept attention depending on the concept budget.

B Language-Model Evaluation Details

This appendix specifies the large-scale CGA-LM experiment in Section 3.2. The goal is to match the PoST language-model evaluation protocol while changing only the sequence mixer. All models are pretrained as base language models and evaluated without instruction tuning or in-context examples.

B.1 Training protocol

Table 3: CGA-LM training protocol for the large language-model evaluation.

Parameter	Value
Training data	FineWeb-Edu [Lozhkov et al., 2024]
Training context length T_{train}	2,048
Tokenizer	Llama-3.1 tokenizer [Dubey et al., 2024]
Hardware	8× H200
Optimizer	AdamW [Loshchilov and Hutter, 2019]
Warmup	1% of total steps
Precision	BF16 mixed precision
Gradient clipping	$\|\nabla\|_2 \leq 1.0$
Training tokens (180M / 440M)	4B / 9B
Learning rate (180M / 440M)	6×10^{-4} / 3×10^{-4}, cosine to 10^{-5}

B.2 State-matched CGA configuration

The CGA state is matched to the recurrent state size used by the PoST Mamba-2 configurations. For Mamba-2, the per-layer runtime state is $d_{\text{inner}} d_{\text{state}}$. For CGA-LM, the online state is CHd_v^c. We therefore use $C = 128$ and $d_v^c = 64$ at both scales:

Table 4: State-matched CGA-LM configurations. Concept identities are fixed layer parameters; the online state is only the concept value memory.

Parameter	**180M**	**440M**
d_{model}	768	1,024
Number of layers	24	48
Heads H	24	32
Concepts C	128	128
Concept value width d_v^c	64	64
CGA online state CHd_v^c	196,608	262,144
Matched Mamba-2 state $d_{\text{inner}} d_{\text{state}}$	1,536×128	2,048×128
Concept identities	fixed random unit addresses	fixed random unit addresses
Read/write addressing	untied projections	untied projections
Retention	trainable, PoST-style parameterization	trainable, PoST-style parameterization

B.3 Evaluation protocol

We use the Language Model Evaluation Harness [Gao et al., 2024] in a zero-shot setting. The reported tasks are LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, WinoGrande, and OpenBookQA, matching Table 2. For reproducibility and systems comparison, each checkpoint should report: validation loss, validation perplexity, zero-shot task metrics, training tokens per second, prefill latency, decode latency, parameter count, online state size, and CGA diagnostics for write gate, read scale, retention, and decay.

NeurIPS Paper Checklist

This file is retained only for the conference-template branch of `main.tex`.

- **Claims.** The abstract and introduction state the paper's scope as a method proposal: CGA replaces token-indexed attention memory with a fixed bank of runtime concept slots.
- **Limitations.** Section 5 states the main limitations, including kernel optimization, retention tuning, concept-count selection, and the absence of a large-scale SOTA language-model claim.
- **Theory.** Section 2 gives the state definition, read/write equations, state-size accounting, and backpropagation boundary for fixed-bank CGA.
- **Experiments.** Section 3 centers on MQAR as the controlled recall benchmark and CGA-LM as the sequence-modeling scaling target.